SESSION 4

Layout Design using CSS

Display Properties · Flexbox · Navigation Bars · Cards · Sections · Footers

Display & Flexbox | Navbars | Cards | Sections | Footers

Prepared & Presented by

Utkarsh Kushwaha

Technical Trainer | Frontend Development Bootcamp 2026

About the Presenter

Utkarsh Kushwaha

Full Stack Developer | Technical Speaker | Community Mentor

Experienced in building responsive and interactive web applications using modern technologies and actively involved in technical communities, workshops, and student learning initiatives.

TECHNICAL SKILLS

HTML | CSS | JavaScript | React | Node.js | MongoDB | Express | Docker

Connect: LinkedIn GitHub

Learning Objectives

By the End of This Session, Students Will Be Able To:

- Understand how display properties control element behavior
- Build flexible layouts using Flexbox container & item properties
- Design a fully responsive navigation bar from scratch
- Create reusable card components for content display
- Structure a webpage using clear, section-based layout blocks
- Design a clean, well-organized footer section

Outcome: Students will be able to design structured, responsive, real-world webpage layouts using CSS.

Session Agenda

Display Properties
block, inline, inline-block, none

Flexbox Fundamentals
container & item properties

Navigation Bar Design
building a responsive navbar

Card Components
reusable content cards

Section-Based Layouts
structuring page content

Footer Design
clean closing sections

Outcome: Build structured and visually organized webpage layouts.

Quick Revision

Previously Learned in Session 3

- CSS Syntax & Selectors
- Colors & Backgrounds
- Typography & Text Styling
- Borders, Margin & Padding
- The CSS Box Model
- Styling a Profile Card (Mini Project)

CODE

```
.profile-card {
  width: 280px;
  background: #1f2937;
  border-radius: 12px;
  padding: 20px;
  box-sizing: border-box;
}

h2 {
  font-family: Arial, sans-serif;
  color: #f5f7fa;
  text-transform: uppercase;
}
```

Outcome: Today we move from styling single elements to arranging whole pages.

01

CSS Display Properties

Controlling How Elements Behave on the Page

The display Property

Every HTML element has a default display behavior — CSS lets you change it.

block

Takes full width, starts on a new line

`div, p, h1, section`

inline

Flows within text, no new line, no width/height

`span, a, strong`

inline-block

Flows inline but accepts width & height

`buttons, badges`

none

Removes the element completely from layout

`hidden menus, modals`

CODE

```
span { display: inline-block; }    li { display: block; }    .menu { display: none; }
```

Outcome: Choose the right display type to control how elements take up space.

02

Flexbox Fundamentals

The Most Important Layout Tool You'll Use Every Day

Flexbox: Container Properties

Applied to the parent element to arrange its children.

`display: flex;`	Turns an element into a flex container
`flex-direction`	row (default) or column — sets main axis
`justify-content`	Aligns items along the main axis
`align-items`	Aligns items along the cross axis
`flex-wrap / gap`	Wraps items onto new lines & adds spacing

CODE

```
.container { display: flex; justify-content: space-between; gap: 16px; }
```

Outcome: Use flex container properties to control how a whole row or column of items is arranged.

Flexbox: Item Properties

Applied to the children inside a flex container.

`flex-grow`	How much an item grows to fill space
`flex-shrink`	How much an item shrinks if space is tight
`flex-basis`	The starting size of an item
`align-self`	Overrides align-items for one item
`order`	Changes visual order without changing HTML

CODE

```
.card { flex: 1; align-self: flex-start; order: 2; }
```

Outcome: Use flex item properties to fine-tune how each individual element behaves inside the layout.

Flexbox in Practice

A 3-box row that adapts using only Flexbox.

CODE

```html
<!-- HTML -->
<div class="row">
  <div class="box">1</div>
  <div class="box">2</div>
  <div class="box">3</div>
</div>

/* CSS */
.row {
  display: flex;
  gap: 16px;
}
.box {
  flex: 1;
  background: #232938;
  padding: 20px;
}
```

RESULT

Why flex: 1 matters:

- Every box grows equally to fill the row
- No fixed widths needed — fully responsive
- gap replaces the need for margin hacks

Outcome: With just display: flex, flex: 1 and gap, you can build most simple layouts.

03

Navigation Bar Design

The First Layout Pattern Every Website Needs

Building a Navigation Bar

A navbar is a flex row: logo on one side, links on the other.

LIVE PREVIEW

MyBrand Home About Services Contact

CODE

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 32px;
}

.nav-links { display: flex; gap: 24px; }
```

Outcome: justify-content: space-between is the key pattern for logo-left, links-right navbars.

04

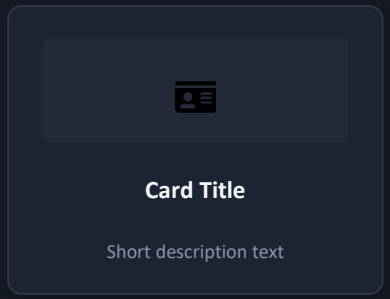
Card Components

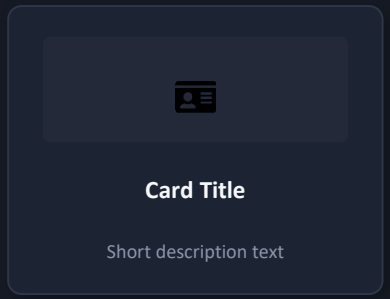
Reusable Building Blocks for Real Webpages

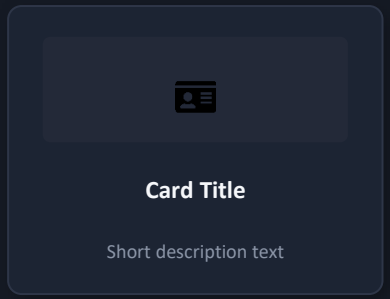
Building a Card Component

Cards group related content into a clean, repeatable box.

LIVE PREVIEW







CSS

```
.card {
  bg: #1c2
  radius: 10px
  shadow: yes
}
```

CODE

```
.card {
  background: #1c2433;
  border-radius: 10px;
  box-shadow: 0 4px 12px rgba(0,0,0,.25);
```

Outcome: One .card class, reused everywhere — the foundation of consistent UI design.

05

Section-Based Layouts

Structuring a Page into Clear, Stacked Blocks

Structuring with Sections

Every webpage is a stack of <section> blocks, each with its own purpose.

PAGE STRUCTURE

<header>

<section class="hero">

<section class="features">

<footer>

CODE

```
<header>...</header>

<section class="hero">
  <h1>Welcome</h1>
</section>

<section class="features">
  ...cards here...
```

CODE

```
section { padding: 60px 24px; max-width: 1200px; margin: 0 auto; }
```

Outcome: Wrap each page idea in its own <section> for clear structure and easy styling.

06

Footer Design

Closing the Page with a Clean, Informative Footer

Designing a Footer

A footer organizes closing info into flex columns.

LIVE PREVIEW

MyBrand

Building the web,

one page at a time.

Links

Home

About

Contact

Follow

GitHub

LinkedIn

Instagram

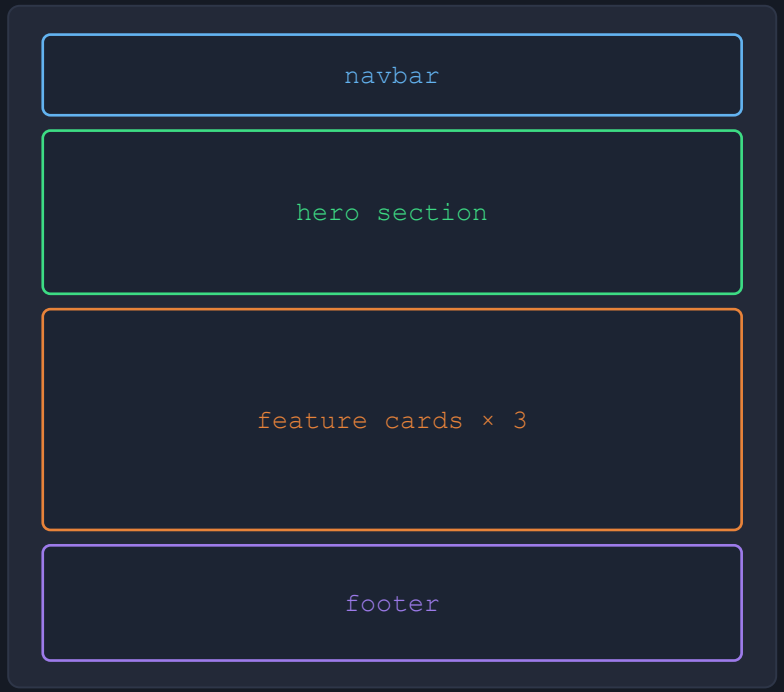
CODE

```
footer {
  display: flex;
  justify-content: space-around;
  flex-wrap: wrap;
  gap: 24px;
}
```

Outcome: flex-wrap keeps footer columns responsive when the screen gets narrow.

Mini Project: Landing Page Layout

Combine everything from today into one structured page.



Build this layout using:

- A flexbox navbar with a logo and links
- A hero section with a heading & button
- Three feature cards in a flex row
- A footer with flex-wrap for responsiveness

Concepts used:
display, flexbox, sections, cards, footer

Best Practices in CSS Layout

Use Flexbox over floats

Floats are outdated for layout — Flexbox is simpler and more reliable

Always set a gap

Use gap instead of margin hacks between flex items

Use semantic sections

header, section, footer over generic divs where possible

Keep components reusable

Write one .card or .navbar class, reuse it everywhere

Mobile-first thinking

Use flex-wrap so layouts adapt to smaller screens

Consistent spacing scale

Stick to a few padding/margin values, not random numbers

Today We Learned

Display Properties
block, inline, inline-block, none

Flexbox
container & item properties

Navigation Bar
logo + links with flex

Card Components
reusable content blocks

Section Layouts
structured page blocks

Footer Design
responsive closing section

Outcome: You can now structure a full webpage layout, ready for the next session.

Thank You!

Questions? I'm happy to help!

Practice Task:

Build a landing page layout with a flexbox navbar, a hero section, three feature cards in a row, and a responsive footer — reuse everything covered today.

Session 2 – next Saturday · HTML Foundations completes — final concepts to get you project-ready